M2. Three Branches of the Membership Tree

Category B Practice - Problem 2 (Basic to Intermediate)

Approach

HonorsStudentMember extends StudentMember, which extends LibraryMember. FacultyMember branches directly from LibraryMember. instanceof classification checks the deepest subtype first; totals rely on polymorphic getBooksBorrowed().

Java Solution

```
class LibraryMember {
    protected final String memberId; protected final int borrowLimit;
    protected int booksBorrowed;
    public LibraryMember(String id,int limit) {
        if(id==null||id.trim().length()<4) throw new IllegalArgumentException("Invalid ID");
        if(limit<=0) throw new IllegalArgumentException("Limit must be positive");
        memberId=id; borrowLimit=limit;
    }
    public void borrowBook() { if(booksBorrowed>=borrowLimit) throw new IllegalStateException("Limit reached"); booksBorrowe
    public int getBooksBorrowed() { return booksBorrowed; }
    public String displayInfo() { return "General Member | Books Borrowed: " + booksBorrowed; }
}
class StudentMember extends LibraryMember {
    protected final String course;
    public StudentMember(String id,int limit,String course) { super(id,limit); this.course=course; }
    @Override public String displayInfo() { return "Student Member | Course: " + course + " | Books Borrowed: " + booksBorro
}
class HonorsStudentMember extends StudentMember {
    private final int bonusLimit;
    public HonorsStudentMember(String id,int limit,String course,int bonus) { super(id,limit,course); bonusLimit=bonus; }
    @Override public String displayInfo() { return "Honors Student Member | Course: " + course + " | Bonus Limit: " + bonusL
}
class FacultyMember extends LibraryMember {
    private final String department;
    public FacultyMember(String id,int limit,String department) { super(id,limit); this.department=department; }
    @Override public String displayInfo() { return "Faculty Member | Department: " + department + " | Books Borrowed: " + bo
}
class MembershipSummary {
    static String classifyGeneration(LibraryMember member) {
        if(member instanceof HonorsStudentMember) return "Multilevel descendant (3 generations deep)";
        if(member instanceof FacultyMember) return "Hierarchical sibling (independent branch)";
        if(member instanceof StudentMember) return "Direct subclass of LibraryMember";
        return "Base class";
    }
    static int getTotalBooksBorrowed(LibraryMember[] members) {
        int total=0; for(LibraryMember member:members) total+=member.getBooksBorrowed(); return total;
    }
}
```

Sample Output

```
classifyGeneration(honors) -> Multilevel descendant (3 generations deep)
classifyGeneration(faculty) -> Hierarchical sibling (independent branch)
Total borrowed: 2 + 1 + 3 = 6
```